

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 5-Rekursif



Arsyaghali Nafiras

109082500068

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1.Rekursif

Rekursi adalah sebuah teknik dalam pemrograman dan matematika di mana sebuah fungsi **memanggil dirinya sendiri** untuk menyelesaikan masalah yang lebih besar dengan cara membaginya menjadi sub-masalah yang lebih kecil dan identik.

Komponen Utama Rekursi:

- Base Case (Kondisi Berhenti)
- Recursive Step (Langkah Rekursif)

B. Guided

1. Guided-1_Membuat baris bilangan dari n hingga 1

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    baris(n)
}

func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Print(bilangan)
        baris(bilangan - 1)
    }
}
```

Output :

```
5
54321
```

2. Guided-2_Menghitung hasil penjumlahan 1 hingga n

```
package main

import "fmt"

func main() {
    var n int

    fmt.Scan(&n)

    fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n - 1)
    }
}
```

Output :

```
5
15
PS C:\ALPR
- modul 5\
4
10
```

3. Guided-3_Mencari dua pangkat n

```

package main

import "fmt"

func main() {

var n int

fmt.Scan(&n)

fmt.Println(pangkat(n))

}

func pangkat(n int) int {

if n == 0 {

return 1

} else {

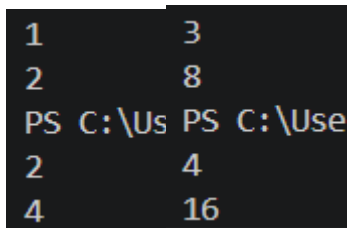
return n * pangkat(n-1)

}

}

```

Output :



```

1      3
2      4
PS C:\Us PS C:\Use
2      4
4      16

```

C. Unguided

- 1) Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

Jawaban :

```

package main

import "fmt"

func fibonacci(n int) int {
    // Base Case: kondisi berhenti
    if n == 0 {
        return 0
    }
    if n == 1 {
        return 1
    }

    // Recursive Step: memanggil diri sendiri sesuai rumus  $S_n = S_{n-1} + S_{n-2}$ 
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    // Menampilkan deret Fibonacci dari suku ke-0 hingga ke-10
    fmt.Println("n | Sn")
    fmt.Println("-----")
    for i := 0; i <= 10; i++ {
        fmt.Printf("%-2d | %d\n", i, fibonacci(i))
    }
}

```

Output :

n		Sn

0		0
1		1
2		1
3		2
4		3
5		5
6		8
7		13
8		21
9		34
10		55

Penjelasan :

- Fungsi fibonacci(n) akan terus memanggil dirinya sendiri dengan nilai n yang semakin mengecil hingga menyentuh angka 1 atau 0.
- $\text{fibonacci}(3) = \text{fibonacci}(2) + \text{fibonacci}(1)$

fibonacci(2) akan diurai lagi menjadi fibonacci(1) + fibonacci(0)

Setelah mencapai *base case* (1 dan 0), nilainya akan dijumlahkan kembali ke atas.

- 2) Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N. Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Jawaban :

```
package main

import "fmt"

func tampilkanFaktor(n int, i int) {
    if i > n {
        return
    }

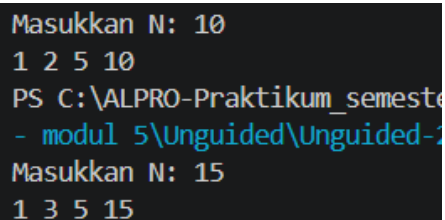
    if n%i == 0 {
        fmt.Printf("%d ", i)
    }

    tampilkanFaktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)

    tampilkanFaktor(n, 1)
    fmt.Println()
}
```

Output :



```
Masukkan N: 10
1 2 5 10
PS C:\ALPRO-Praktikum_semesta>
- modul 5\Unguided\Unguided-2
Masukkan N: 15
1 3 5 15
```

Penjelasan :

- `fmt.Scan(&n)`: Mengambil input dari pengguna.
- `n % i == 0`: Operator modulo digunakan untuk mengecek sisa pembagian. Jika hasilnya 0, maka i adalah faktor dari n.

- Fungsi tampilkanFaktor akan terus memanggil dirinya sendiri sampai nilai i melewati n. fungsi ini menggantikan perulangan (looping) biasa.

3) Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N. Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N .

Jawaban :

```
package main
import "fmt"

func tampilkanGanjil(current int, n int) {
    if current > n {
        return
    }

    fmt.Printf("%d ", current)
    tampilkanGanjil(current+2, n)
}

func main() {
    var n int

    fmt.Scan(&n)

    tampilkanGanjil(1, n)
    fmt.Println()
}
```

Output :

```
10
1 3 5 7 9
PS C:\ALPRO-Praktikum
- modul 5\Unguided\Un
13
1 3 5 7 9 11 13
```

Penjelasan :

- Input: Program menerima angka N.
- Titik Awal: Fungsi rekursif dimulai dari angka 1.
- Kondisi Berhenti (Base Case): Jika angka yang sedang diproses sudah lebih besar dari N, fungsi berhenti.
- Proses Rekursi:

Cetak angka saat ini.

Panggil kembali fungsi tersebut dengan menambah angka sebesar 2 (agar tetap ganjil: 1 3 5).

D. Kesimpulan

- Pengertian Rekursif

Rekursif adalah sebuah teknik pemrograman di mana suatu subprogram (fungsi atau prosedur) memanggil dirinya sendiri. Teknik ini digunakan untuk menyelesaikan masalah besar dengan cara memecahnya menjadi sub-masalah yang identik.

Rekursif merupakan alternatif dari struktur kontrol perulangan (iteratif).

- Komponen Utama Rekursif

Base-case (Basis): Kondisi di mana proses rekursif harus berhenti. Ini adalah bagian terpenting yang harus ditentukan pertama kali sebelum membuat program.

Recursive-case: Bagian di mana subprogram memanggil dirinya sendiri dengan parameter yang bergerak mendekati base-case. Kondisi ini merupakan negasi atau komplemen dari base-case.

- Alur Kerja Rekursif

Forward (Maju): Subprogram terus memanggil dirinya sendiri hingga kondisi *base-case* terpenuhi.

Backward (Mundur): Setelah *base-case* tercapai, program akan kembali ke pemanggil sebelumnya untuk menyelesaikan instruksi yang tersisa hingga kembali ke pemanggilan awal.